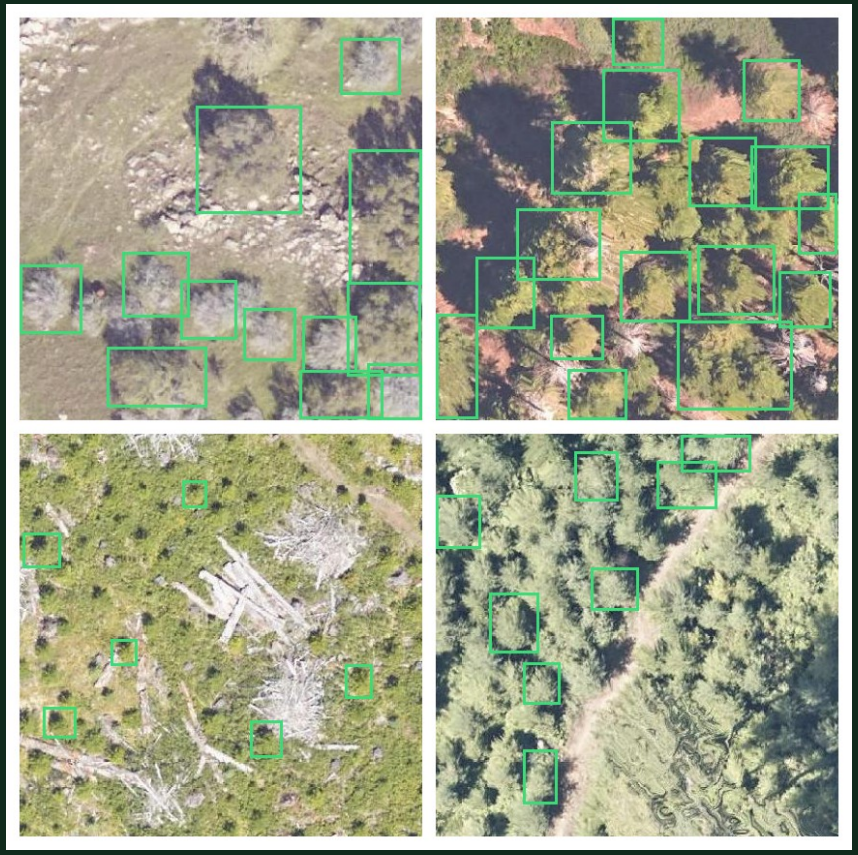
MINI PROJECT · PROJECT REVIEW

Tree Counting Using Drone Images

Automatic detection and counting of tree canopies from aerial imagery using YOLOv11



Nitheesh · Nishu Kumari · Astley

Guided by our project mentor

THE PROBLEM

Counting trees by hand does not scale

- **Slow.** A field survey of a single plantation block can take days of manual walking and tallying.
- **Expensive.** Forestry and agriculture teams spend most of the survey budget on labour, not analysis.
- **Inconsistent.** Two surveyors counting the same plot rarely agree, and nobody can audit the result later.
- **Not repeatable.** Yearly monitoring means paying the full cost again, every single year.

WHO NEEDS THIS

- Forestry and plantation management
- Agriculture and orchard yield planning
- Environmental and carbon monitoring
- Urban green cover surveys

Drones already fly over these sites. The images exist. What is missing is a system that reads them.

OBJECTIVES

What we set out to build

01	**Detect**	Locate every individual tree canopy in a top-view drone image with a bounding box.
02	**Count**	Turn those detections into a reliable total count per image and per surveyed area.
03	**Train**	Train and tune a YOLOv11 detector on a real annotated aerial dataset.
04	**Evaluate**	Measure quality with standard detection metrics plus a counting accuracy metric.
05	**Ship**	Leave behind a reproducible pipeline that a non-author can clone and rerun.

OUR APPROACH

Detection first, counting second

We treat tree counting as an object detection problem, not a regression problem.

Density regression

Predicts a count directly from the image.

Gives you a number and nothing else. You cannot see which trees it found, so you cannot debug it or trust it.

Object detection (our choice)

Predicts a box per tree, then counts the boxes.

The count is explainable. Every tree is visible on the image, mistakes are visible too, and the same output feeds mapping and health analysis later.

WHY YOLOv11

Single stage

One forward pass per image, fast enough for large image batches

Small objects

Anchor-free head handles the dense, small canopies in our data

Mature tooling

Ultralytics gives training, validation and export out of the box

Deployable

Nano and small variants can run on modest hardware in the field

DATASET

Tree-Top-View, an annotated aerial dataset

1,362
annotated aerial images

11,377
labelled tree canopies

8.4
trees per image on average

1
class: tree-top

SPLIT USED FOR TRAINING

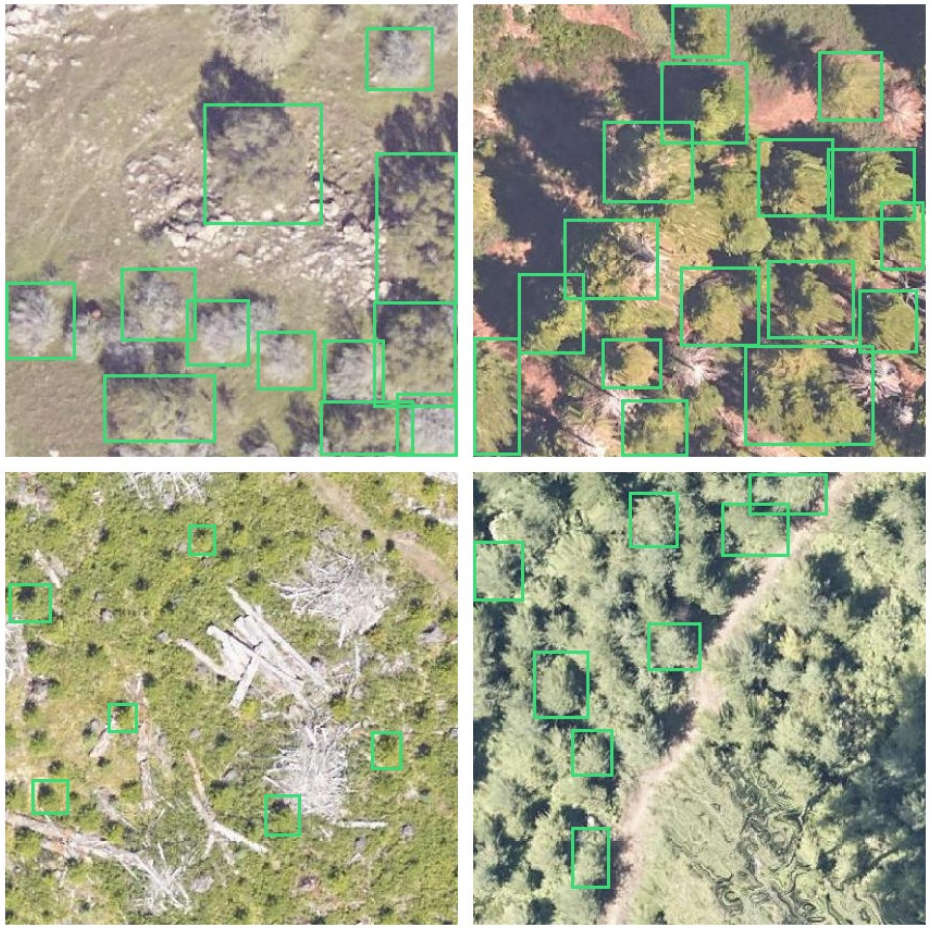


AT A GLANCE

Source	**Roboflow Universe**
Format	**YOLOv11**
Resolution	**416 x 416 px**
Licence	**CC BY 4.0**
Scenes	**Forest, plantation, mixed terrain**

DATASET SAMPLES

What the model has to learn



Four validation images with their ground truth boxes

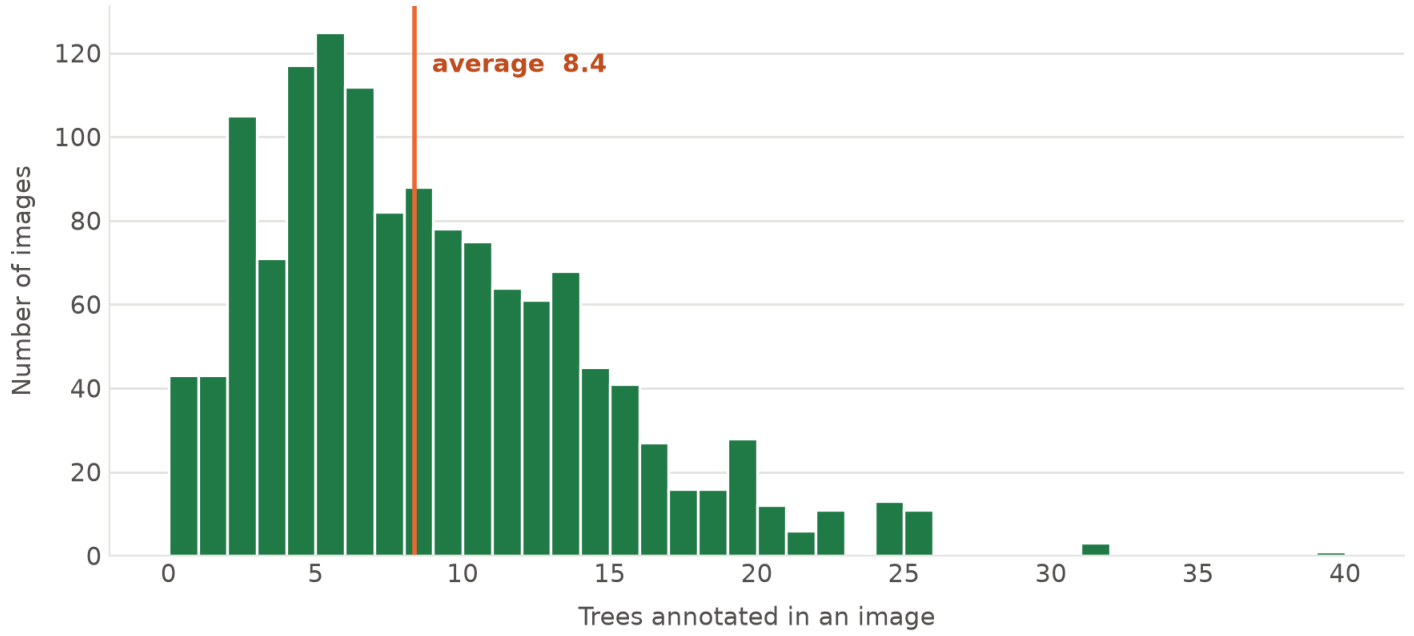
- Canopies overlap and touch, so boxes overlap too
- Tree colour is close to grass and shrub colour
- Shadows and bare ground look like canopies
- Scale varies widely inside a single image
- Some scenes are dense forest, others are sparse

This is why a naive colour or blob threshold fails, and why a trained detector is worth the effort.

DATA ANALYSIS

How many trees are in a typical image

DISTRIBUTION OF ANNOTATIONS PER IMAGE



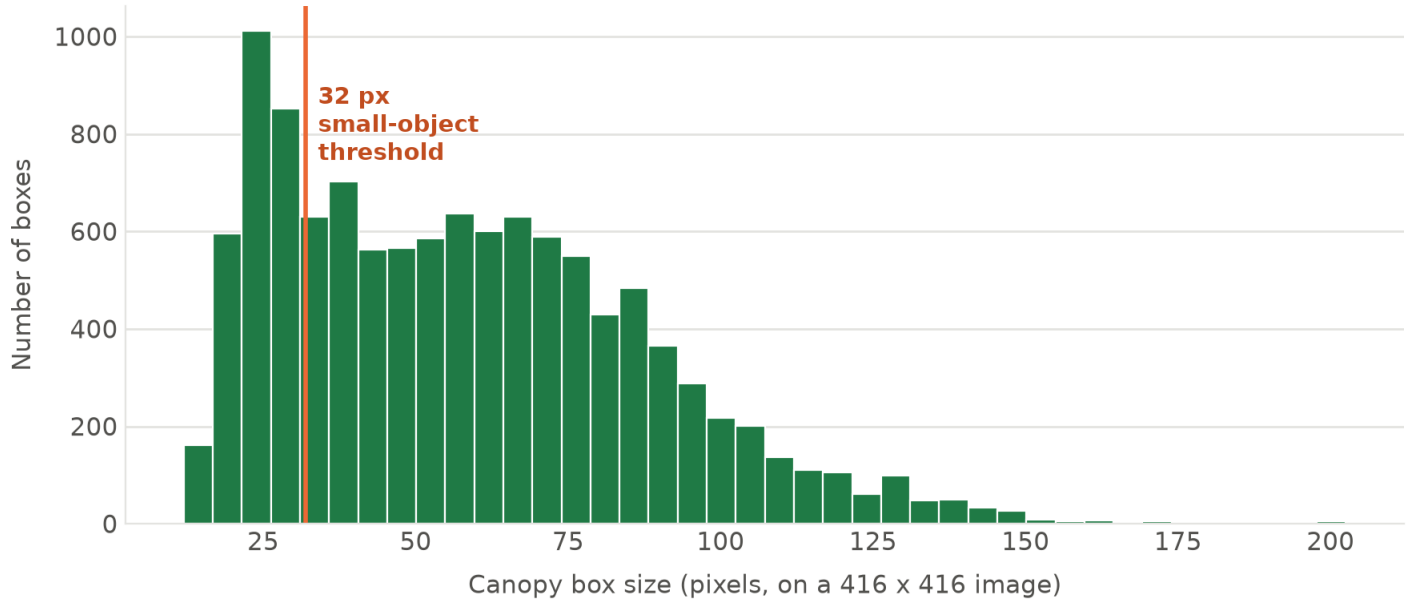
WHAT WE READ FROM IT

- Most images hold 3 to 15 trees
- The tail runs to 40 in one image
- 43 images have no trees at all, useful as negative examples
- Counting error will scale with density, so we report it by band

DATA ANALYSIS

The hard part is that trees are small

CANOPY BOX SIZE DISTRIBUTION



55 px
median canopy width on a 416 px image

24%
of all boxes fall under the 32 px small-object threshold

A quarter of our targets are small objects. That directly shapes our training plan: higher input resolution and small-object augmentation.

DATA ANALYSIS

One finding we want to flag early

The 1,191 training images are not 1,191 distinct scenes.

Roboflow applied offline augmentation before export. Counting unique base filenames shows the real picture:

1,191
training image files

375
unique source scenes

~3x
augmented copies per scene

WHY IT MATTERS

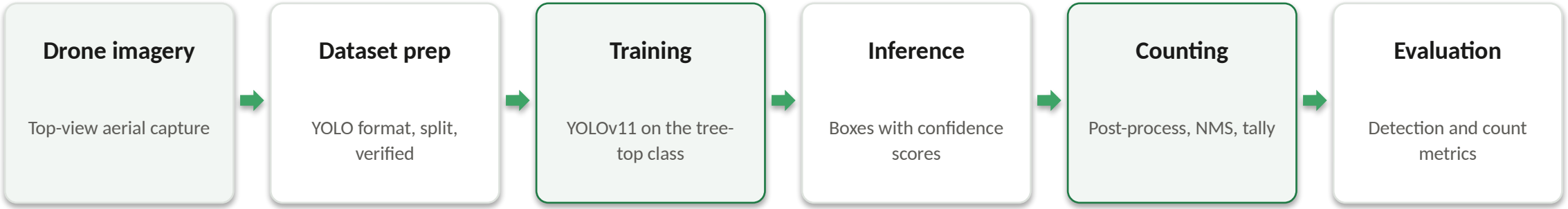
Random splitting would leak augmented twins of a training scene into validation and inflate our scores.

WHAT WE WILL DO

Verify that the provided splits are scene-disjoint, and report metrics on the untouched test set only.

METHOD

End to end pipeline



TRAINING CONFIGURATION WE START FROM

```
yolo detect train \
  model=yolo11n.pt \
  data=datasets/Tree-Top-View/data.yaml \
  epochs=100  imgsz=640
```

- **Baseline first.** yolo11n gives us a fast reference run.
- **Then scale.** yolo11s and yolo11m if accuracy demands it.
- **imgsz 640.** Upscaled from 416 to help the small canopies.

EVALUATION PLAN

How we will judge the result

Detection quality and counting quality are different questions, so we measure both.

DETECTION METRICS

Precision
Of the trees we detected, how many were real

Recall
Of the real trees, how many we found

mAP@0.5
Detection quality at a standard overlap threshold

mAP@0.5:0.95
Detection quality averaged over strict thresholds

COUNTING AND PRACTICAL METRICS

Count MAE
Average absolute difference between predicted and true count

Count error %
Relative counting error, reported per density band

Inference speed
Milliseconds per image, for field feasibility

Failure review
Manual look at the worst images, not just the averages

ENGINEERING SETUP

The repository is already structured for this

```
TreeCountingUsingDroneImages/
├── datasets/  Tree-Top-View: train, valid, test, data.yaml
├── src/       train, predict, evaluate, visualize, utils
├── models/    checkpoints and trained weights
├── outputs/   metrics, predictions, visualizations
├── notebooks/ exploration and analysis
├── scripts/   reproducible run scripts
├── docs/      report and presentation
└── requirements.txt
```

- **Version controlled.** Every change tracked on GitHub from day one.
- **Separated concerns.** Data, code, weights and outputs never mix.
- **Reproducible.** Clone, install requirements, run one command.
- **Documented.** README carries setup, dataset and workflow.

Stack: Python 3 · PyTorch · Ultralytics YOLOv11 · OpenCV · NumPy · Pandas · Matplotlib · Git

PROGRESS

Where we are today

Repository setup	Done
Dataset collection	Done
Dataset analysis	Done
Literature survey	In progress
Model training	Next
Hyperparameter tuning	Planned
Evaluation	Planned
Final documentation	Planned

Data work is complete and understood. The next milestone is a trained baseline with real numbers to show.

RISKS

What could go wrong, and our plan for it

Small canopies are missed	24% of boxes are under 32 px, the classic weak spot for detectors.	Train at 640 or 960 input size, add scale and mosaic augmentation, tune the confidence threshold for recall.
Overlapping canopies get merged	Dense forest means boxes sit on top of each other and NMS may delete true positives.	Tune the NMS IoU threshold explicitly and validate the count, not just the mAP.
Limited unique scenes	375 distinct source scenes is a modest amount of real variety.	Rely on augmentation, keep the test split untouched, and report honestly rather than overclaiming.
Domain shift on new drone data	A different camera, altitude or season may not look like our training set.	Document the operating conditions the model was trained for, and flag it as future work.

NEXT STEPS

What happens next

Now	**Verify split integrity** Confirm no augmented twin of a training scene appears in valid or test.
Week 1	**Baseline training run** Train yolo11n for 100 epochs and record the first full metric set.
Week 2	**Tune and compare** Sweep image size, augmentation and model size, then pick a winner.
Week 3	**Counting layer** Build the count post-processing and measure counting error directly.
Week 4	**Report and demo** Annotated prediction gallery, final metrics, written report.

Ask for the mentor: is our evaluation plan complete, and should we prioritise accuracy or field-deployable speed?

THE TEAM

Thank you

Nitheesh	Project lead, model development, repository, documentation
Nishu Kumari	Literature survey, dataset analysis, documentation
Astley	Data collection, research, model evaluation, testing

Dataset: Tree-Top-View, Roboflow Universe, CC BY 4.0 · Built with Ultralytics YOLOv11 and PyTorch